SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA10	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	**AKTCHAYA A**	**Reg. No.:**	**192424096**
Date:	**03/08/2026**	**Duration:**	60 minutes

Code of Conduct

I AKTCHAYA A certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: ___________________________

Annexure A – Architecture Design Assignment

Instructions to Students:
Each student will be assigned an **individual software project problem statement**. Based on the assigned problem, analyze the system requirements and design an appropriate software architecture by applying software engineering principles. Your submission should demonstrate your ability to select, justify, and evaluate a suitable architectural style.

Question

Based on your **assigned problem statement**, perform an **Architecture Design Analysis** and prepare an architecture design report by answering the following questions:

1. Analyze System Requirements

• Study the given problem statement and identify the system objectives.

• Analyze the functional and non-functional requirements that influence the software architecture. • Identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.

2. Select a Suitable Software Architecture

• Select an appropriate architectural style for the proposed system (e.g., Layered Architecture, Client-Server, MVC, Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture). • Justify why the selected architecture is the most suitable for the given problem.

3. Draw the Software Architecture Diagram

• Design a clear architecture diagram illustrating the overall structure of the proposed system. • Include all major layers/components, databases, external systems, APIs, and communication mechanisms. • Use appropriate software architecture notations and labels.

4. Explain Components and Their Interactions

- Describe the purpose and responsibilities of each architectural component.
- Explain how the components communicate and exchange data.
- Illustrate the overall workflow of the system.

5. Discuss Quality Attributes

Evaluate how the proposed architecture addresses the following aspects:

- Scalability
- Security
- Performance
- Reliability
- Maintainability
- Availability

6. Justify Architectural Decisions

- Explain the rationale behind your architectural choices.
- Discuss the trade-offs considered while selecting the architecture.
- Explain how the chosen architecture supports future enhancements, system integration, and long-term maintainability.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.
Architecture Diagram and Component Design	Produces a clear, complete, and well structured architecture diagram with all major components, interfaces, and interactions accurately represented.	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and lacks sufficient detail.	Diagram is incomplete, unclear, or technically incorrect.

Component Interaction and Quality Attribute Analysis	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.
Architectural Justification and Technical Presentation	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well organized, professional report.	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope; report needs improvement.	Justification is weak or absent; report lacks organization and technical clarity.

Criteria	Mark
System Requirement Analysis	/5
Selection of Software Architecture	/5
Architecture Diagram and Component Design	/5
Component Interaction and Quality Attribute Analysis	/5
Architectural Justification and Technical Presentation	/5
TOTAL	/5

ARCHITECTURE DESIGN ASSIGNMENT

UNIVERSITY ACADEMIC MANAGEMENT AND STUDENT SUCCESS PLATFORM

1. Analyze System Requirements

- **System Objectives**
  - Manage all student academic information through one centralized platform.
  - Automate student admissions and course registration processes.
  - Monitor attendance, examinations, and academic performance.
  - Maintain complete academic records for every student.
  - Identify academically at-risk students using institutional policies.
  - Notify students, faculty, and advisors about important academic events.
  - Generate academic and institutional reports for decision-making.
  - Improve communication between students, faculty, and administrators.
  - Support data-driven academic planning.
  - Provide a scalable and secure academic management system.
- **Functional Requirements**
  - Register new students into the university system.
  - Manage student profiles and academic records.
  - Allow students to register for courses online.
  - Manage faculty information and teaching schedules.
  - Record and monitor student attendance.
  - Conduct examinations and store examination results.
  - Calculate grades and academic performance.

- Identify students who are academically at risk.
- Send notifications to students, faculty, and academic advisors.
- Generate attendance, performance, and institutional reports.
- Manage departments, programs, and academic resources.
- Provide dashboards for students, faculty, and administrators.

- **Non-Functional Requirements**
  - The system should support thousands of users simultaneously.
  - The platform should provide high availability with minimum downtime.
  - The system should protect academic data using strong security mechanisms.
  - The application should provide fast response times.
  - The platform should be easy to maintain and update.
  - The system should support future expansion without redesign.
  - The platform should ensure reliable data storage and backup.
  - The application should provide role-based access control.
  - The system should maintain data accuracy and consistency.
  - The platform should support integration with future university services.
- **Quality Attributes**
  - **Scalability:** The system should support increasing students and departments.
  - **Maintainability:** The system should allow easy updates and modifications.
  - **Reliability:** The platform should provide accurate and consistent academic data.
  - **Availability:** The system should be accessible throughout the academic year.
  - **Performance:** The platform should process requests quickly.
  - **Security:** The system should secure confidential student and faculty information.

2. Select a Suitable Software Architecture

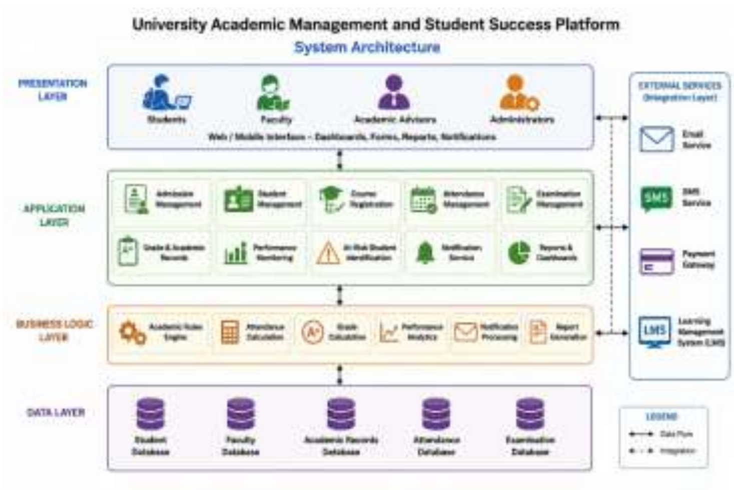
Selected Architecture: Layered Architecture (Three-Tier Architecture)

Justification

- Layered Architecture separates presentation, business logic, and data management.
- It simplifies system development and maintenance.
- It supports multiple user roles such as students, faculty, and administrators.
- It improves system security through controlled access.
- It allows independent modification of each layer.
- It supports future integration with external university services.
- It is suitable for large educational management systems.
- It improves code reusability and scalability.

3. Software Architecture Diagram

You can recreate the following simple architecture diagram in **Draw.io, Visio, PowerPoint, Lucidchart, or Canva** for your assessment.



4. Explain Components and Their Interactions

- **Presentation Layer**
    - Provides user interfaces for students, faculty, advisors, and administrators.
    - Receives user requests and displays system responses.

- **Admission Management**
    - Manages student admissions and enrollment.
- **Student Management**
    - Maintains student profiles and academic information.
- **Course Registration**
    - Allows students to register and manage courses.
- **Attendance Management**
    - Records daily attendance and calculates attendance percentages.
- **Examination and Grade Management**
    - Stores examination schedules, marks, and final grades.
- **Academic Performance Monitoring**
    - Monitors student progress using academic data.
- **At-Risk Student Identification**
    - Detects students with poor attendance or low academic performance.
- **Notification Service**
    - Sends alerts and reminders to students, faculty, and advisors.
- **Report and Dashboard Module**
    - Generates academic reports and visual dashboards.
- **Business Logic Layer**
    - Processes academic rules and institutional policies.

- **Database Layer**
    - Stores all academic, attendance, faculty, and examination information.
- **External Services**
    - Provides email, SMS, payment, and LMS integration.
- **Workflow**
    - Users log in to the platform.
    - Students register for courses.
    - Faculty record attendance and examination marks.
    - The business layer processes academic rules.
    - Student performance is continuously analyzed.
    - At-risk students are identified automatically.
    - Notifications are sent to students and faculty.
    - Reports and dashboards are generated for administrators.

5. Discuss Quality Attributes

- **Scalability:** The architecture supports thousands of students and multiple departments.
- **Security:** Role-based authentication protects academic information.
- **Performance:** Layer separation enables faster request processing.
- **Reliability:** Centralized databases ensure accurate and consistent records.
- **Maintainability:** Individual layers can be updated without affecting other layers.
- **Availability:** The platform provides continuous access to academic services.

6. Justify Architectural Decisions

- **Rationale**
    - Layered Architecture provides a clear separation of responsibilities.
    - The architecture simplifies development, testing, and maintenance.
    - The design improves security by controlling access between layers.

- The architecture supports centralized academic data management.
- The solution enables efficient communication among all stakeholders.

- **Trade-offs**
  - Layered Architecture may introduce additional processing between layers.
  - Large systems may require optimization to reduce communication overhead.
  - Initial development effort is higher because each layer is designed separately.
- **Support for Future Enhancements**
  - New academic modules can be added without changing the entire system.
  - External university systems can be integrated through APIs.
  - The architecture supports cloud deployment and future scalability.
  - Additional analytics and AI-based student success modules can be integrated.
  - The modular design ensures long-term maintainability and institutional growth.